WayDine — Build Plan

Oct 29, 2026 · @User

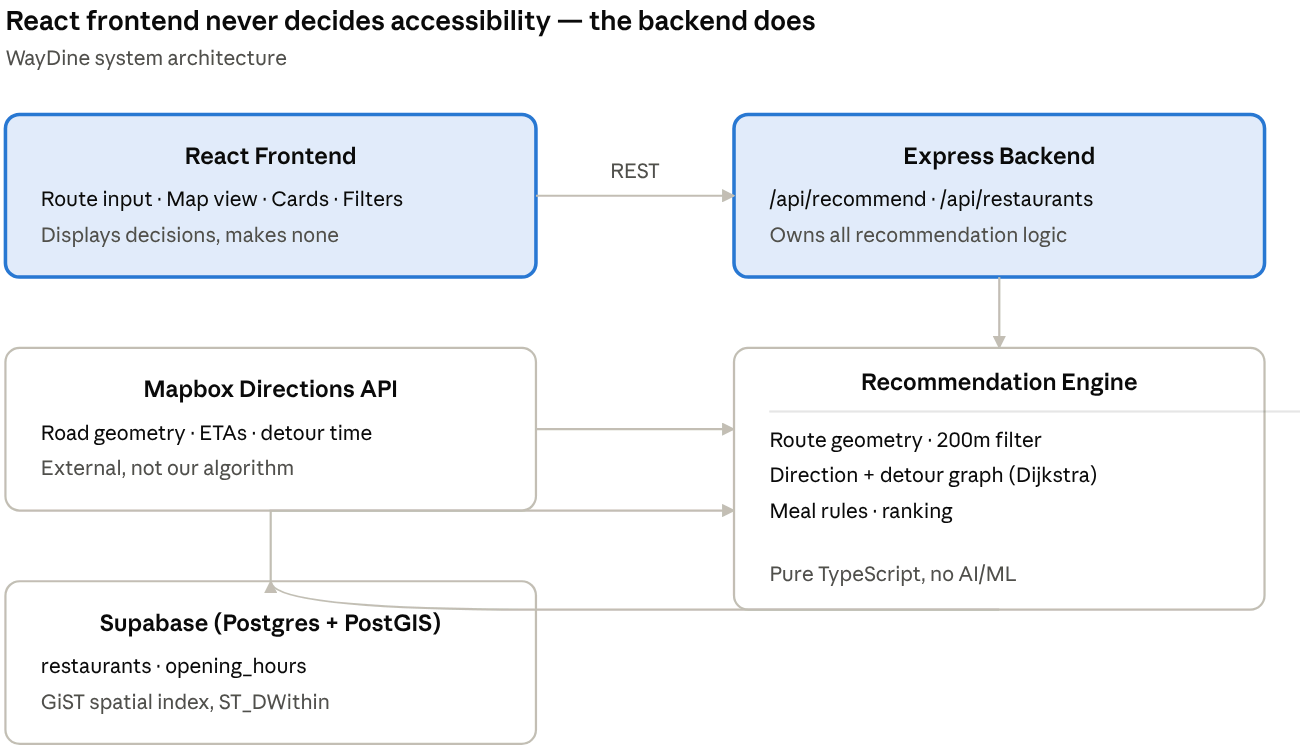
A to Z build plan for the WayDine hackathon project — tech stack, architecture, database, backend, recommendation engine, frontend/PWA setup, and execution schedule.

1. Tech stack — what goes where and why

Layer	Technology	Used for	Why this one
Frontend framework	React + Vite + TypeScript	All UI: search form, map view, result cards, details drawer	Fast dev server, no SSR complexity needed for a 6-hr build
Styling	Tailwind CSS + shadcn/ui	Component styling, cards, drawers, buttons	Pre-built accessible components, no custom CSS from scratch
Maps / rendering	Mapbox GL JS	Interactive map, route polyline, restaurant pins	Industry-standard, free tier covers hackathon usage
Routing / ETA	Mapbox Directions API	Real driving route geometry, travel durations, waypoint detours	We do not build our own road router — Mapbox supplies the road network and ETAs, same as any production app
Backend runtime	Node.js + Express	REST API: `/api/recommend`, `/api/restaurants`, `/api/health`	Shared JS/TS with frontend, minimal boilerplate
Database	Supabase (hosted Postgres)	All structured data: restaurants, hours, search history	Relational data (FKs, joins) fits better than a document store

Layer	Technology	Used for	Why this one
Geospatial queries	PostGIS (Supabase extension)	`ST_DWithin` to find restaurants within 200m of route polyline	Native spatial indexing (GiST), avoids app-side distance math
Graph algorithm layer	Plain TypeScript (no library)	Dijkstra / DAG shortest-path over the candidate-restaurant graph, to rank stops by true detour cost	This is the "graph theory" requirement from the problem statement — it runs on real Mapbox-derived edge weights, not a toy example
Direction/side-of-road math	Plain TypeScript	Cross-product classification (left/right of travel vector)	Cheap, deterministic, no AI/ML
Deployment	Vercel (frontend) + Render (backend)	Hosting both halves for the live demo	Free tier, fast deploys, no DevOps overhead
PWA layer	Vite PWA plugin (`vite-plugin-pwa`)	Manifest + service worker so the app installs on a phone home screen	Near-zero config on top of existing Vite setup

What we are deliberately not using: no Python backend, no Next.js, no auth system, no microservices, no custom road-network graph (we don't have OSM data for this), no AI/ML model anywhere in the pipeline (explicitly banned by the problem statement).



system architecture · frontend, backend, Mapbox, Supabase

The frontend only renders what the backend decides. All recommendation logic — filtering, direction, ranking — lives in one place: `/api/recommend`.

2. API keys and environment setup

Key / credential	Where it's used	Env var name	Notes
Mapbox access token (public)	Frontend — Mapbox GL JS map rendering	`VITE_MAPBOX_TOKEN`	Create at mapbox.com, free tier; scope it to URL restrictions before the demo
Mapbox access token (secret, optional)	Backend — Directions API calls, if routed server-side to hide quota usage	`MAPBOX_SECRET_TOKEN`	Optional — simplest path is calling Directions API directly from frontend with the public token

Key / credential	Where it's used	Env var name	Notes
Supabase project URL	Frontend + backend — DB client init	`VITE_SUPABASE_URL` (frontend), `SUPABASE_URL` (backend)	From Supabase project settings
Supabase anon/public key	Frontend — read-only queries if frontend ever queries directly	`VITE_SUPABASE_ANON_KEY`	Safe to expose; respects RLS policies
Supabase service role key	Backend only — full DB access from Express	`SUPABASE_SERVICE_ROLE_KEY`	**Never expose this in frontend code or commit it**

Setup checklist:

1. Create Mapbox account → generate access token → add to frontend `.env` as `VITE_MAPBOX_TOKEN`.
2. Create Supabase project → enable PostGIS extension (Database → Extensions → postgis) → copy URL + keys into both `.env` files.
3. Backend `.env` (never committed, in `.gitignore`):

```
SUPABASE_URL=
SUPABASE_SERVICE_ROLE_KEY=
PORT=4000
```

4. Frontend `.env`:

```
VITE_MAPBOX_TOKEN=
VITE_SUPABASE_URL=
VITE_SUPABASE_ANON_KEY=
VITE_API_BASE_URL=
```

5. Add `.env` and `.env.local` to `.gitignore` before the first commit — four teammates means four chances to leak a key to a public repo.

3. Database schema (Supabase / Postgres + PostGIS)

Three tables only — `road_nodes` / `road_edges` are deliberately cut (see tech stack note); we don't have OSM data to populate them in 6 hours, and the graph layer runs on the candidate-restaurant set instead, not a fake road network.

restaurants

Column	Type	Notes
id	uuid, PK	`gen_random_uuid()`
name	text	
latitude	double precision	
longitude	double precision	
geo_location	geography(Point, 4326)	generated from lat/lng, GiST-indexed
highway_id	text	which corridor this sits on (e.g. `NH48-mumbai-pune`)
side	text	`left` or `right`, relative to the standard direction of travel on that highway
cuisine	text	
veg_available	boolean	
road_access_verified	boolean	manually checked for the demo set — flagged honestly as not a live data feed

opening_hours

Column	Type	Notes
id	uuid, PK	
restaurant_id	uuid, FK → restaurants.id	
weekday	int	0=Sunday .. 6=Saturday
opens_at	time	

Column	Type	Notes
closes_at	time	handle rows that cross midnight in app logic, not schema
meal_type	text	`breakfast` / `lunch` / `snacks` / `dinner` — merged in from the old `food_availability` table to cut scope

search_history

Column	Type	Notes
id	uuid, PK	
source	geography(Point, 4326)	
destination	geography(Point, 4326)	
departure_time	timestamptz	
preferences	jsonb	vegetarian, max_detour_minutes, etc.
created_at	timestamptz	default `now()`

Setup SQL:

```
create extension if not exists postgis;

create table restaurants (
  id uuid primary key default gen_random_uuid(),
  name text not null,
  latitude double precision not null,
  longitude double precision not null,
  geo_location geography(Point, 4326)
    generated always as (
      ST_SetSRID(ST_MakePoint(longitude, latitude), 4326)::geography
    ) stored,
  highway_id text not null,
  side text check (side in ('left','right')),
  cuisine text,
  veg_available boolean default false,
  road_access_verified boolean default false
);
```

```
create index restaurants_geo_idx on restaurants using gist (geo_location);

create table opening_hours (
  id uuid primary key default gen_random_uuid(),
  restaurant_id uuid references restaurants(id) on delete cascade,
  weekday int check (weekday between 0 and 6),
  opens_at time not null,
  closes_at time not null,
  meal_type text check (meal_type in ('breakfast','lunch','snacks','dinner'))
);

create table search_history (
  id uuid primary key default gen_random_uuid(),
  source geography(Point, 4326),
  destination geography(Point, 4326),
  departure_time timestamptz,
  preferences jsonb,
  created_at timestamptz default now()
);
```

Core query — candidates within 200m of the route polyline:

```
select id, name, latitude, longitude, side, cuisine, veg_available
from restaurants
where ST_DWithin(
  geo_location,
  ST_GeogFromText('LINESTRING(...)'),  -- the route polyline from Mapbox, as
WKT
  200
);
```

Seed 20–30 restaurant rows on one real corridor (Mumbai–Pune, NH48) with a mix of left/right side, varied opening hours, and at least one deliberately excluded case (closed at arrival, or outside 200m) to demonstrate the filters actually work in the demo.

4. Backend structure

Folder layout:

```
backend/
  src/
    index.ts              # Express app entry
    routes/
      recommend.ts         # POST /api/recommend
      restaurants.ts        # GET /api/restaurants
      health.ts             # GET /api/health
    services/
      mapbox.ts             # Directions API wrapper
      candidates.ts          # Supabase + PostGIS query (200m filter)
      direction.ts           # cross-product side-of-road classification
      graph.ts                # candidate graph build + Dijkstra/DAG
shortest-path
      mealRules.ts             # arrival time -> meal type
      ranking.ts                # weighted scoring + exclusion filters
    db/
      supabaseClient.ts
    types/
      index.ts              # shared request/response types
  .env
  package.json
```

Endpoints:

Endpoint	Method	Purpose
`/api/recommend`	POST	Main pipeline: route → candidates → direction → graph ranking → meal rules → response
`/api/restaurants`	GET	Raw seeded restaurant list, for debugging/admin view
`/api/health`	GET	Returns `{status: "ok"}`, used for deploy verification

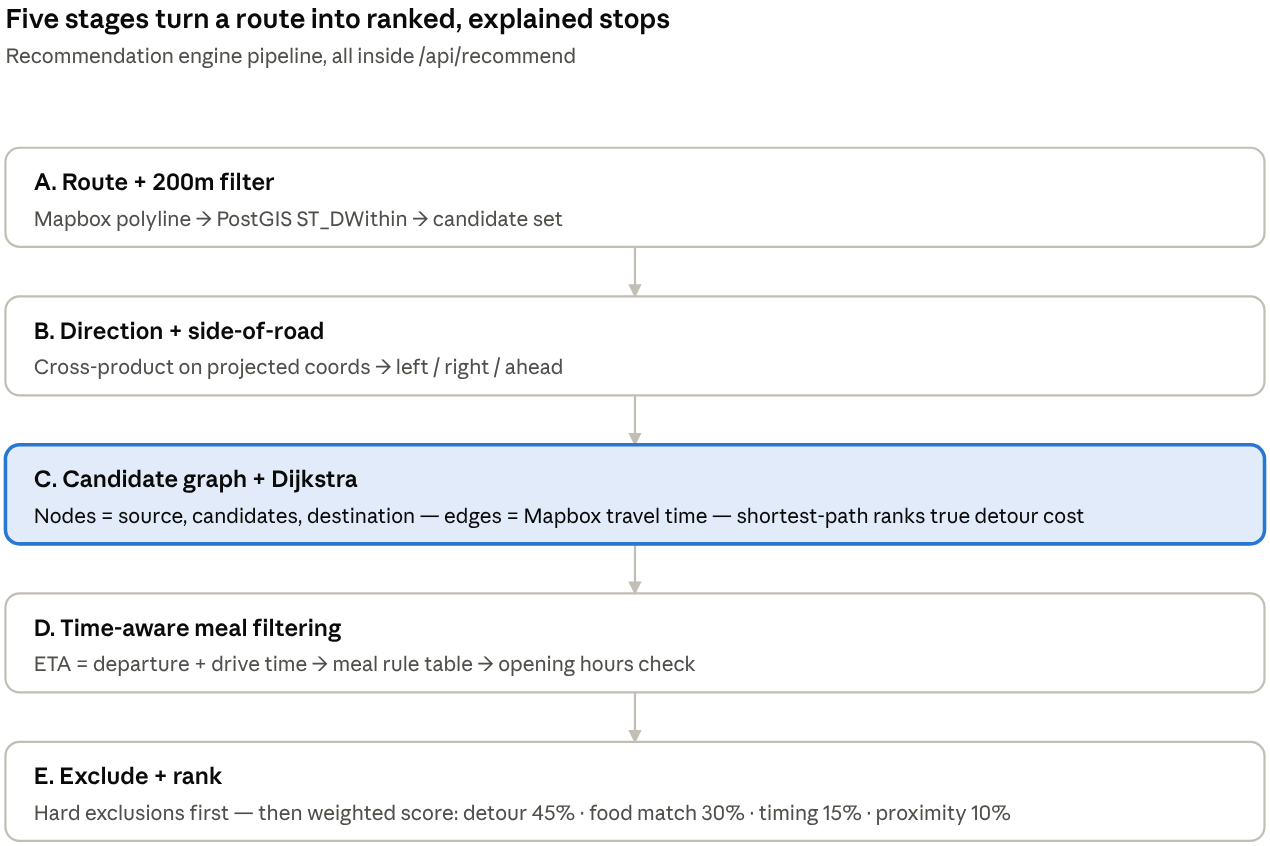
`/api/recommend` request shape (lock this with the team before coding — Hour 1):

```
{
  "source": [72.8777, 19.0760],
  "destination": [73.8567, 18.5204],
  "departureTime": "2026-10-30T10:00:00+05:30",
  "preferences": {
    "vegetarian": true,
    "maxDetourMinutes": 10
  }
}
```

Response shape:

```
{
  "route": { "polyline": [...], "totalDurationMin": 142 },
  "recommendations": [
    {
      "id": "uuid",
      "name": "Highway Spice Kitchen",
      "etaArrival": "12:45",
      "detourMinutes": 3,
      "distanceFromRoadM": 120,
      "side": "right",
      "mealRecommended": "lunch",
      "openAtArrival": true,
      "score": 0.87,
      "reason": "On your driving direction, open on arrival, minimal detour"
    }
  ],
  "excluded": [
    { "id": "uuid", "name": "...", "reason": "Closed at expected arrival
time" }
  ]
}
```

Including `excluded` with reasons is what powers the "No Results" UI state honestly — it shows *why* something was filtered instead of silently disappearing.



recommendation engine pipeline · 5 stages

Stage C is the graph-theory core and the part most likely to get questioned — be precise about it:

Build a directed graph where nodes are the route's start, every surviving candidate restaurant, and the destination. Edge weight between two nodes = Mapbox-derived driving time between them (not straight-line distance). Run Dijkstra (or a DAG shortest-path, since candidates are ordered along the route — faster and still legitimate) to compute, for each candidate:

```
ΔT = T(source → restaurant → destination) - T(source → destination)
```

This is real graph theory on real data — Mapbox supplies the road network and edge weights (same as any app would use), and the algorithm is ours. Do not build a second "toy" graph just to demonstrate Dijkstra separately; that invites the exact question we're trying to avoid ("is the algorithm doing real work or is it decorative?").

5. Frontend structure and PWA setup

Folder layout:

```
frontend/
  src/
    main.tsx
    App.tsx
    components/
      SearchForm.tsx        # source, destination, preferences, submit
      MapView.tsx             # Mapbox GL map, route line, pins
      RecommendationCard.tsx   # single restaurant result
      ResultsPanel.tsx          # list of cards, left sidebar
      DetailsDrawer.tsx          # expanded restaurant details
      NoResultsState.tsx          # shows excluded[] reasons
    hooks/
      useRecommend.ts            # calls POST /api/recommend
    lib/
      supabaseClient.ts
      mapbox.ts
    types/
      index.ts                    # mirrors backend response types
  public/
    icons/                          # PWA icons, see below
    manifest.webmanifest
  vite.config.ts
  .env
```

Single-page layout: sidebar (search + results) on the left, map filling the rest. Expandable details drawer instead of separate routes/pages — faster to build, fewer states to manage under time pressure.

Making it a PWA (installable on a phone home screen):

1. Install the plugin:

```
npm install -D vite-plugin-pwa
```

2. In `vite.config.ts`:

```
import { VitePWA } from 'vite-plugin-pwa';

export default {
  plugins: [
```

```
    VitePWA({
      registerType: 'autoUpdate',
      manifest: {
        name: 'WayDine',
        short_name: 'WayDine',
        description: 'Eat on the way, not out of the way',
        theme_color: '#2F4F2F',
        background_color: '#FAF7F0',
        display: 'standalone',
        start_url: '/',
        icons: [
          { src: '/icons/icon-192.png', sizes: '192x192', type: 'image/png'
},
          { src: '/icons/icon-512.png', sizes: '512x512', type: 'image/png' }
        ]
      }
    })
  ]
};
```

3. Add two icon sizes (192x192, 512x512 PNG) to `public/icons/` — even a quick logo works for a hackathon demo.
4. Build + serve over HTTPS (Vercel does this automatically) — service workers require HTTPS except on localhost.
5. Verify installability: open the deployed URL on a phone in Chrome → "Add to Home Screen" should appear, or check Lighthouse's PWA audit in desktop Chrome DevTools.

This is genuinely low-effort — maybe 20 minutes total — and it's a tangible extra to show judges: "this runs as an installed app, not just a browser tab."

6. Execution plan

Hour	Focus	Deliverable
1	Foundation	Supabase + PostGIS enabled, schema created, 20–30 seed rows, Mapbox account + token, repo scaffolded, `/api/recommend` request/response shape locked by the whole team

Hour	Focus	Deliverable
2	Route + candidates	Mapbox Directions integration working end-to-end, map renders polyline, `ST_DWithin` candidate query returns real results
3	Graph + direction	Candidate graph built, Dijkstra/DAG shortest-path computing ΔT per candidate, cross-product direction classification working
4	Time + ranking	Meal-time rules, opening-hours check, exclusion filters, weighted ranking, `excluded[]` reasons in response
5	UI integration	Sidebar, map markers, result cards, details drawer, no-results state, basic PWA setup
6	Hardening	Fix integration bugs, deploy both services, rehearse 2 demo scenarios, no new features after this point starts

Team split (4 people):

Member	Owns
1	UI/UX — React components, sidebar, cards, drawer
2	Mapbox integration — route geometry, map rendering, polyline display
3	Graph algorithms — candidate graph, Dijkstra/DAG shortest-path, direction + ranking logic
4	Supabase + Express — schema, PostGIS queries, endpoints, deployment

Two demo scenarios to rehearse (not a feature tour):

1. **The opposite-side trap:** two restaurants equally close to the highway, one on the correct side with a clean approach, one requiring a U-turn — show only the correct one gets recommended, and why.

2. **The detour-cost catch:** a restaurant very close to the road (e.g. 80m) but requiring a 15-minute loop to actually reach — show it's excluded or ranked low despite proximity, because the graph layer computed real detour time, not straight-line distance.

Non-negotiable cuts if time runs short (in priority order): PWA polish → custom fonts/theme → details drawer (fold into card) → `search_history` logging → never cut: the 200m filter, direction classification, the graph/Dijkstra layer, and meal-time rules — those four are what the problem statement is actually scoring.